

ALEXANDRE Ewan, Groupe 1
26/01/2026

TD3 SLAM : JDBC

1) Problématique	1
2) Etapes	1
3) Résultat avec SceneBuilder	9
4) Conclusion	11

1) Problématique

Nous cherchons à apprendre à faire communiquer une application Java avec une base de données MySQL, pour cela nous allons effectuer plusieurs étapes en utilisant BlueJ, XAMPP et le driver JDBC.

2) Etapes

1. Tout d'abord, pour effectuer ce TD il va falloir installer le driver JDBC à partir de ce lien : <https://dev.mysql.com/downloads/connector/j/>

Lorsqu'on y est, il faut sélectionner Platform Independent :

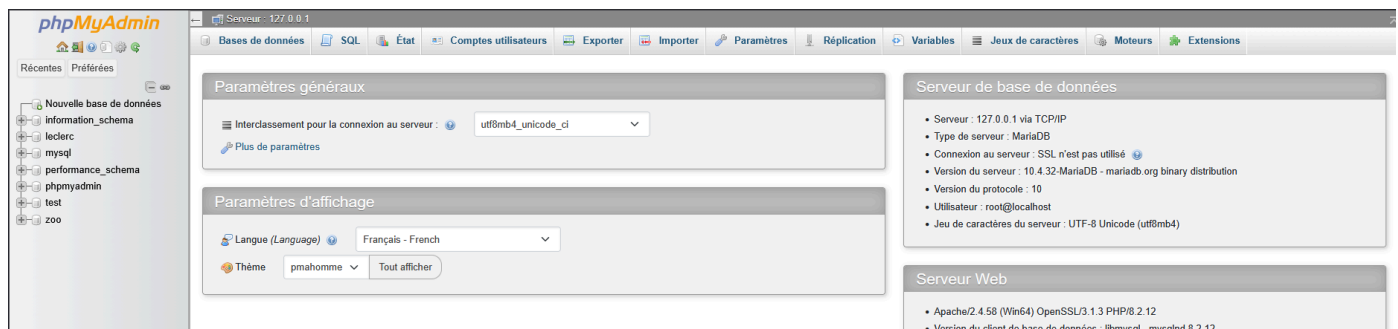


Et installer le ZIP :

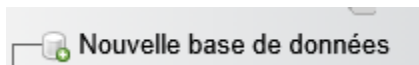
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-9.6.0.zip)	9.6.0	5.1M	Download
MD5: cebec7bc546020cd03ad56c293b83eb6 Signature			

Lorsque c'est fait nous allons ouvrir BlueJ, puis nous allons aller dans **Outils > Préférences > Bibliothèques** puis cliquer sur **Add File** et ajouter le **mysql-connector-j-9.6.0.jar**. Lorsque c'est fait, il suffit de redémarrer BlueJ et on sera prêt à commencer les prochaines étapes.

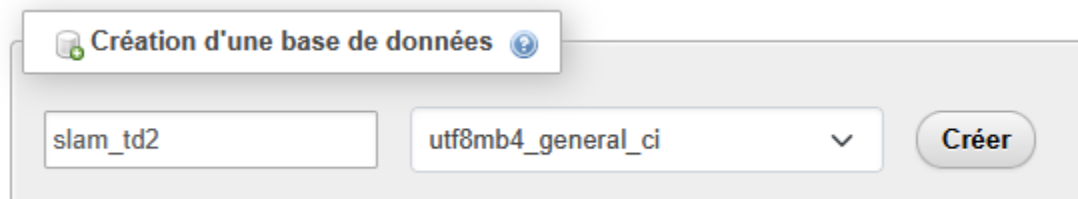
2. Maintenant nous allons démarrer le phpmyadmin avec XAMPP en sélectionnant Apache et MySQL, en cliquant sur Start pour les deux et en cliquant sur Admin sur la partie MySQL pour ouvrir le phpmyadmin :



3. Pour créer la base de donnée nous allons cliquer sur



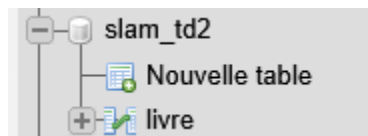
Puis on va créer la BDD slam_td2 :



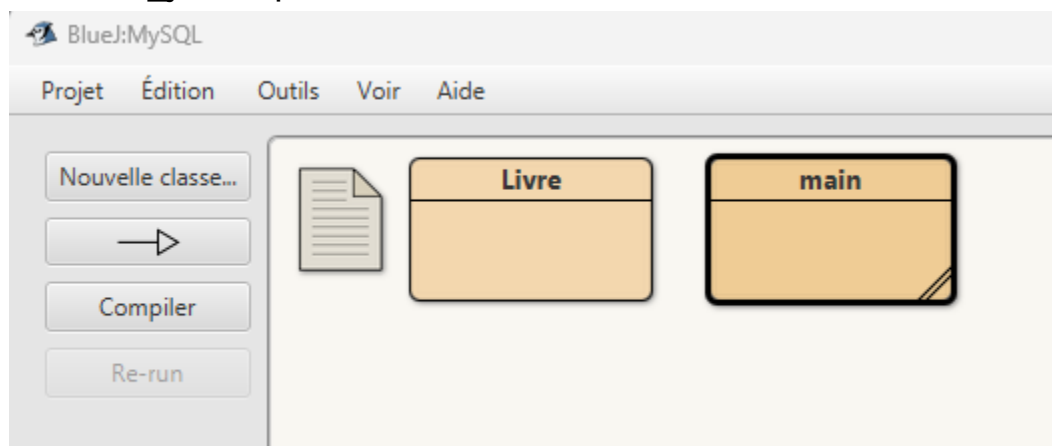
Ensuite nous allons aller dans la section **Importer** et ajouter le fichier **td3_bdd.sql** :



Cela devrait créer la table livre dans la BDD :



4. Afin de pouvoir créer un exécutable nous allons tout d'abord créer un projet MySQL dans lequel nous allons ajouter les deux .java présent dans td1_java.zip :



Lorsque c'est fait nous allons remplacer le code de la classe main avec ce code pour pouvoir créer une structure de stockage :

```

import java.sql.*;
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        // Paramètres de connexion XAMPP
        String url = "jdbc:mysql://localhost:3306/slam2_td2";
        String login = "root";
        String passwd = "";

        // Initialisation de la collection (Question 4)
        ArrayList<Livres> maBibliotheque = new ArrayList<>();

        Connection cn = null;
        Statement st = null;
        ResultSet rs = null;

        try {
            // Étape 1 : Chargement du driver (nécessite le .jar dans BlueJ)
            Class.forName("com.mysql.jdbc.Driver");

            // Étape 2 : Connexion à la base de données
            cn = DriverManager.getConnection(url, login, passwd);

            // Étape 3 : Création du Statement
            st = cn.createStatement();

            // Étape 4 & 5 : Récupération des livres de la BDD
            String sql = "SELECT * FROM livres";
            rs = st.executeQuery(sql);

            while (rs.next()) {
                // Création de l'objet Livres à partir de la ligne SQL
                Livres l = new Livres(
                    rs.getString("ISBN"),
                    rs.getString("titre"),
                    rs.getString("auteur"),
                    rs.getInt("prix")
                );
                // Ajout à la bibliothèque (Question 5)
                maBibliotheque.add(l);
            }

            // Affichage pour vérifier
            System.out.println("Contenu de la bibliothèque (" + maBibliotheque.size() + " livres) :");
            for (Livres l : maBibliotheque) {
                l.Afficher();
                System.out.println("");
            }
        } catch (SQLException e) {
            System.out.println("Erreur SQL : " + e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println("Driver introuvable ! Vérifiez les bibliothèques BlueJ.");
        } finally {
            // Fermeture propre des ressources
            try {
                if (rs != null) rs.close();
                if (st != null) st.close();
                if (cn != null) cn.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

```

Maintenant nous démarrons le Main, cela nous montre bien que cela fait le lien avec la BDD et qu'il est exécutable :

```
Contenu de la bibliothèque des livres :
Titre: Le monde s effondre, Auteur: Chinua Achebe, Prix: 29 €
Titre: Contes, Auteur: Hans Christian Andersen, Prix: 15 €
Titre: Orgueil et Prejuges, Auteur: Jane Austen, Prix: 29 €
Titre: Le Pere Goriot, Auteur: Honore de Balzac, Prix: 19 €
Titre: Decameron, Auteur: Boccace, Prix: 16 €
Titre: Fictions, Auteur: Jorge Luis Borges, Prix: 14 €
Titre: Les Hauts de Hurlevent, Auteur: Emily Bront e, Prix: 5 €
Titre: L etranger, Auteur: Albert Camus, Prix: 20 €
Titre: Voyage au bout de la nuit, Auteur: Louis-Ferdinand Celine, Prix: 13 €
Titre: Don Quichotte, Auteur: Miguel de Cervantes, Prix: 15 €
Titre: Les Contes de Canterbury, Auteur: Geoffrey Chaucer, Prix: 9 €
```

5. Nous allons maintenant assurer le remplissage en modifiant le code du Main :

```
1 import java.sql.*;
2 import java.util.ArrayList;
3
4 public class Main {
5     public static void main(String[] args) {
6         // Paramètres de connexion adaptés à XAMPP et votre nom de base
7         String url = "jdbc:mysql://127.0.0.1:3306/slam_td2";
8         String login = "root";
9         String passwd = "";
10
11         // Question 4 : Initialisation de la collection
12         ArrayList<Livres> maBibliotheque = new ArrayList<>();
13
14         Connection cn = null;
15         Statement st = null;
16         ResultSet rs = null;
17
18         try {
19             // Etape 1 : Chargement du driver
20             Class.forName("com.mysql.jdbc.Driver");
21
22             // Etape 2 : Récupération de la connexion
23             cn = DriverManager.getConnection(url, login, passwd);
24
25             // Etape 3 : Création d'un statement
26             st = cn.createStatement();
27
28             // Etape 5 : Exécution de la requête pour récupérer tous les livres
29             String sql = "SELECT * FROM livres";
30             rs = st.executeQuery(sql);
31
32             // Parcours du résultat et remplissage de la liste (Question 5)
33             while (rs.next()) {
34                 // On extrait les données de la ligne SQL actuelle
35                 String isbn = rs.getString("ISBN");
36                 String titre = rs.getString("titre");
37                 String auteur = rs.getString("auteur");
38                 int prix = rs.getInt("prix");
39
40                 // On crée l'objet Livres et on l'ajoute à la bibliothèque
41                 Livres l = new Livres(isbn, titre, auteur, prix);
42                 maBibliotheque.add(l);
43
44                 // Affichage pour confirmer le chargement
45                 l.Afficher();
46                 System.out.println("");
47             }
48
49             System.out.println("Nombre total de livres chargés : " + maBibliotheque.size());
50
51         } catch (SQLException e) {
52             e.printStackTrace();
53         } catch (ClassNotFoundException e) {
54             System.out.println("Erreur : Driver JDBC non trouvé dans BlueJ.");
55         } finally {
56             // Fermeture des ressources
57             try {
58                 if (rs != null) rs.close();
59                 if (st != null) st.close();
60                 if (cn != null) cn.close();
61             } catch (SQLException e) {
62                 e.printStackTrace();
63             }
64         }
65     }
66 }
```

Comme on peut le voir cette fois le code vérifie bien le remplissage :

```
Titre: L Amour aux temps du cholera, Auteur: Gabriel Garcia Marquez, Prix: 5 €  
Titre: epopee de Gilgamesh, Auteur: Anonyme, Prix: 18 €  
Titre: Faust, Auteur: Johann Wolfgang von Goethe, Prix: 23 €  
Titre: Les ames mortes, Auteur: Nicolas Gogol, Prix: 24 €  
Titre: Le Tambour, Auteur: Gunter Grass, Prix: 21 €  
Nombre total de livres chargés : 33
```

6. A présent nous allons faire en sorte de synchroniser la BDD et le JAVA avec les SET en modifiant la classe Livre :

```
1 import java.sql.*;  
2  
3 public class Livre {  
4     private String ISBN, titre, Auteur;  
5     private int prix;  
6  
7     // Constructeur original (utilisé pour charger les données)  
8     public Livre(String ISBN, String titre, String auteur, int prix) {  
9         this.ISBN = ISBN;  
10        this.titre = titre;  
11        this.Auteur = auteur;  
12        this.prix = prix;  
13    }  
14  
15    // --- Méthodes SET modifiées pour JDBC (Question 6) ---  
16  
17    public void setTitre(String titre, Statement st) {  
18        try {  
19            this.titre = titre;  
20            String sql = "UPDATE livre SET titre = '" + titre + "' WHERE ISBN = '" + this.ISBN + "'";  
21            st.executeUpdate(sql);  
22        } catch (SQLException e) {  
23            e.printStackTrace();  
24        }  
25    }  
26  
27    public void setAuteur(String auteur, Statement st) {  
28        try {  
29            this.Auteur = auteur;  
30            String sql = "UPDATE livre SET auteur = '" + auteur + "' WHERE ISBN = '" + this.ISBN + "'";  
31            st.executeUpdate(sql);  
32        } catch (SQLException e) {  
33            e.printStackTrace();  
34        }  
35    }  
36  
37    public void setPrix(int prix, Statement st) {  
38        try {  
39            this.prix = prix;  
40            // Requête SQL de mise à jour du prix  
41            String sql = "UPDATE livre SET prix = '" + prix + "' WHERE ISBN = '" + this.ISBN + "'";  
42            st.executeUpdate(sql);  
43        } catch (SQLException e) {  
44            e.printStackTrace();  
45        }  
46    }  
47  
48    public void setISBN(String isbn, Statement st) {  
49        try {  
50            String oldISBN = this.ISBN;  
51            this.ISBN = isbn;  
52            String sql = "UPDATE livre SET ISBN = '" + isbn + "' WHERE ISBN = '" + oldISBN + "'";  
53            st.executeUpdate(sql);  
54        } catch (SQLException e) {  
55            e.printStackTrace();  
56        }  
57    }  
58  
59    // --- Méthodes GET et Affichage (inchangées) ---  
60  
61    public String getTitre() { return titre; }  
62    public String getAuteur() { return Auteur; }  
63    public int getPrix() { return prix; }  
64    public String getISBN() { return ISBN; }  
65  
66    public void Afficher() {  
67        System.out.print("Titre: " + titre + ", ");  
68        System.out.print("Auteur: " + Auteur + ", ");  
69        System.out.print("Prix: " + prix + " €");  
70    }  
71 }
```

7. Ensuite nous allons tester cela en ajoutant ce code au Main :

```
for (Livre l : maBibliotheque) {  
    if (l.getISBN().equals(isbnRecherche)) {  
        System.out.println("Livre trouvé ! Ancien prix : " + l.getPrix() + "€");  
  
        // On utilise le setter persistant de la Question 6  
        l.setPrix(10, st);  
  
        System.out.println("Mise à jour réussie. Nouveau prix : " + l.getPrix() + "€");  
        break; // On arrête la recherche une fois trouvé  
    }  
}
```

Résultat :

```
Titre: Faust, Auteur: Johann Wolfgang von Goethe,  
Titre: Les ames mortes, Auteur: Nicolas Gogol, Pr  
Titre: Le Tambour, Auteur: Gunter Grass, Prix: 21  
Nombre total de livres chargés : 33  
Chargement terminé : 33 livres importés.  
Livre trouvé ! Ancien prix : 15€  
Mise à jour réussie. Nouveau prix : 10€
```

Ça fonctionne !

8. Désormais nous allons ajouter un deuxième constructeur qui va permettre d'enregistrer physiquement les livres avec une commande INSERT INTO en modifiant le main :

```
1 import java.sql.*; // Import indispensable pour le Statement et SQLException
2
3 public class Livre {
4     private String ISBN, titre, Auteur;
5     private int prix;
6
7     /**
8      * Premier constructeur (TD1)
9      * Utilisé pour charger les livres existants depuis la BDD vers Java (Question 5)
10     */
11     public Livre(String ISBN, String titre, String auteur, int prix) {
12         this.ISBN = ISBN;
13         this.titre = titre;
14         this.Auteur = auteur;
15         this.prix = prix;
16     }
17
18     /**
19      * QUESTION 8 : Deuxième constructeur (Surcharge)
20      * Permet d'insérer un nouveau livre directement dans MySQL au moment de l'instanciation
21     */
22     public Livre(String ISBN, String titre, String auteur, int prix, Statement st) {
23         this.ISBN = ISBN;
24         this.titre = titre;
25         this.Auteur = auteur;
26         this.prix = prix;
27
28         try {
29             // Requête SQL d'insertion (Attention aux guillemets simples pour les String)
30             String sql = "INSERT INTO livre (ISBN, titre, auteur, prix) VALUES ("
31                 + "'" + ISBN + "', "
32                 + "'" + titre + "', "
33                 + "'" + auteur + "', "
34                 + prix + ")";
35
36             // Exécution de la commande d'insertion
37             st.executeUpdate(sql);
38             System.out.println("Succès : Le livre a été inséré dans la base de données.");
39         } catch (SQLException e) {
40             System.out.println("Erreur lors de l'insertion SQL : " + e.getMessage());
41         }
42     }
43
44     // --- Méthodes SET (Question 6) ---
45     public void setPrix(int prix, Statement st) {
46         try {
47             this.prix = prix;
48             String sql = "UPDATE livre SET prix = " + prix + " WHERE ISBN = '" + this.ISBN + "'";
49             st.executeUpdate(sql);
50         } catch (SQLException e) {
51             e.printStackTrace();
52         }
53     }
54
55     // --- Autres méthodes (GET et Affichage) ---
56     public String getISBN() { return ISBN; }
57     public String getTitre() { return titre; }
58     public int getPrix() { return prix; }
59
60     public void Afficher() {
61         System.out.print("Titre: " + titre + ", ");
62         System.out.print("Auteur: " + Auteur + ", ");
63         System.out.print("Prix: " + prix + " €"); // Symbole euro corrigé
64     }
65 }
```


9. Enfin, c'est l'heure de tester l'ajout d'un livre avec ce code dans le Main :

```
// QUESTION 9 : Ajout du nouveau livre de M. Gravoull
// On utilise le constructeur persistant (Question 8)
System.out.println("Ajout du livre de M. Gravoull...");
Livre livreGravoull = new Livre("999123456", "Coder comme un pro en Java", "M. Gravoull", 99, st);

// On l'ajoute à la bibliothèque locale pour que la liste soit à jour
maBibliotheque.add(livreGravoull);

System.out.println("Terminé ! Nombre total de livres : " + maBibliotheque.size());
```

Ensuite on lance le code avec BlueJ et le message dit que ça fonctionne :

```
Bibliothèque chargée. Taille : 33
Prix mis à jour pour le livre ISBN 1317442277.
Ajout du livre de M. Gravoull...
Succès : Le livre a été inséré dans la base de données.
Terminé ! Nombre total de livres : 34
```

Et comme on peut le voir ça fonctionne :

ISBN	titre	auteur	prix
6447873157	L education sentimentale	Gustave Flaubert	7
9127300501	Romancero gitano	Federico Garcia Lorca	14
6188391185	Cent ans de solitude	Gabriel Garcia Marquez	6
7966181142	L Amour aux temps du cholera	Gabriel Garcia Marquez	5
2248495033	epopee de Gilgamesh	Anonyme	18
7202147245	Faust	Johann Wolfgang von Goethe	23
6875678783	Les ames mortes	Nicolas Gogol	24
2170342861	Le Tambour	Gunter Grass	21
999123456	Coder comme un pro en Java	M. Gravoull	99

3) Résultat avec SceneBuilder

Maintenant que nous savons faire cela sans application, nous allons utiliser l'application faite dans un TD précédent pour pouvoir ajouter les livres dans la base de données avec l'interface graphique. J'ai fait un mélange des deux codes pour que ça puisse donc faire le lien entre l'application et la base de données.

Pour pouvoir tester ça il faut simplement remplir toutes les infos sur l'application :

Ma Bibliothèque SQL

ISBN DU LIVRE: 1234567889

TITRE DU LIVRE: La souffrance de Ewan

PRIX DU LIVRE: 999999

AUTEUR DU LIVRE: ALEXANDRE Ewan

AJOUTER

Nombre de livres en base : 39

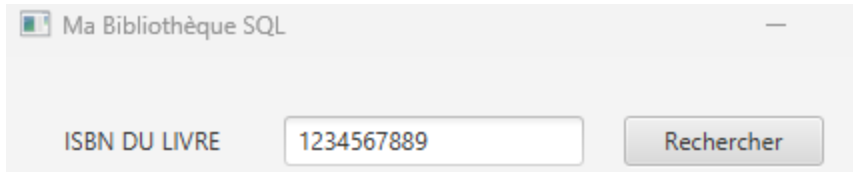
Ensuite on clique sur ajouter, ça va modifier le compteur du bas :

Nombre de livres en base : 40

On s'aperçoit que ça marche :

ISBN	titre	auteur	prix
6447873157	L education sentimentale	Gustave Flaubert	7
9127300501	Romancero gitano	Federico Garcia Lorca	14
6188391185	Cent ans de solitude	Gabriel Garcia Marquez	6
7966181142	L Amour aux temps du cholera	Gabriel Garcia Marquez	5
2248495033	epopee de Gilgamesh	Anonyme	18
7202147245	Faust	Johann Wolfgang von Goethe	23
6875678783	Les ames mortes	Nicolas Gogol	24
2170342861	Le Tambour	Gunter Grass	21
999123456	Coder comme un pro en Java	M. Gravoull	99
1115423546	TEST DU JDBC	Alexandre Ewan	250
8888888888	test	testmagique	90
998448487489489	testtest	4545	100
99999999999	TESTMAGIQUE	JAJA	110
5415644651561	TESTMAGIQUE2	JAJA	110
1234567889	La souffrance de Ewan	ALEXANDRE Ewan	999999

Maintenant on va tester la recherche des livres en utilisant l'isbn en utilisant celui du nouveau livre (1234567889) :



Ma Bibliothèque SQL

ISBN DU LIVRE

Résultat :



Ma Bibliothèque SQL

ISBN DU LIVRE

TITRE DU LIVRE

PRIX DU LIVRE

AUTEUR DU LIVRE

Livre trouvé !

Ça marche !

4) Conclusion

En conclusion j'ai appris à faire le lien entre le JAVA et le SQL en utilisant BlueJ et XAMPP ce qui me permet donc d'utiliser un moyen graphique de modifier mes bases de données et ce qui facilite grandement la tâche.